Q3 - Sonic Signatures: Audio Fingerprinting + Identifier App

Q3A 'Magical Mystery Tune' (7.5%) + Q3B 'Zapptain America' (5%) - EE200 Course Project

This builds a Shazam-style identifier that never compares raw waveforms: it converts audio to a time-frequency picture, keeps only a sparse set of standout peaks as a fingerprint, and matches fingerprints by time alignment. The real provided library of 50 songs is indexed from data/songs/ (filenames kept exactly as given - the filename without extension is the ground-truth label); the full library produces about 2.47 million paired hashes. The pipeline lives in src/q3_fingerprint.py; parameters quoted below (sample rate 11025 Hz, STFT window 1024 with 50% overlap, peak neighbourhood 20 bins, fan-out 15, target-zone gap 1-40 frames) are defined there with justification.

Q3A - Sonic Signatures

1. Why a single Fourier transform is not enough

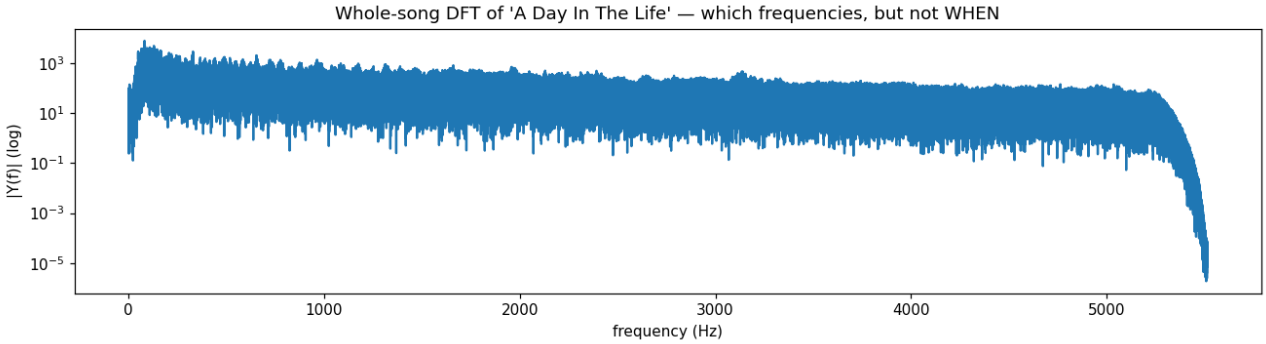

Fig 1. DFT magnitude of an entire song: it reveals WHICH frequencies are present, but not WHEN.

The DFT of a whole song tells you which frequencies the song contains but completely discards timing - the same set of notes in any order gives a similar magnitude spectrum. Since recognition depends on the pattern of frequencies OVER TIME, a single DFT is insufficient. We need to watch the frequency content evolve as the song plays.

2. The spectrogram and the time-frequency resolution trade-off

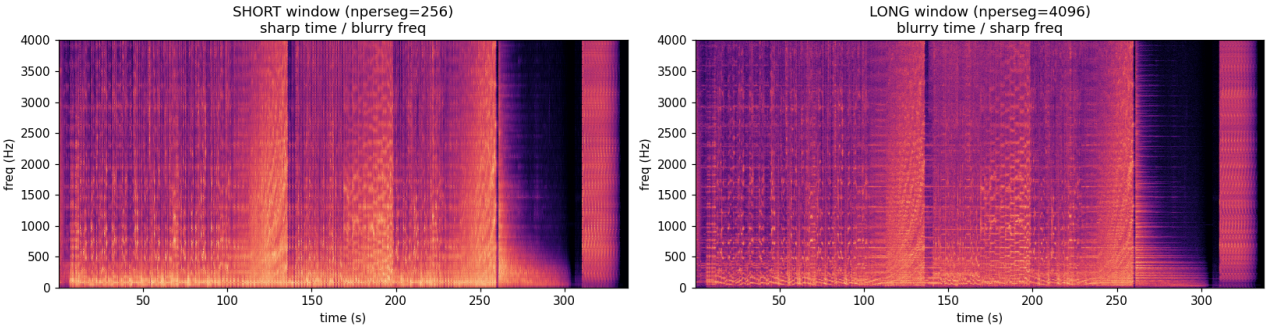

Fig 2. Spectrogram of the song with a SHORT window (left) and a LONG window (right).

A spectrogram restores timing: slide a short window along the signal and take the DFT of each windowed slice, then stack the slices side by side so time runs horizontally, frequency vertically, and brightness shows how strong each frequency is at each instant (a rising note traces a rising streak, a steady tone a horizontal line). Each column is just an ordinary DFT of one short piece. Recomputing with a short vs. a long window exposes the resolution trade-off: a SHORT window localises events sharply in time but smears them in frequency (wide, blurry bands); a LONG window resolves frequency finely but smears events in time. One cannot have both at once - the same uncertainty principle seen in Q2(c). The fingerprinter picks an intermediate window (1024 samples at 11025 Hz, ~93 ms with 50% overlap) to balance the two.

3. The fingerprint: a constellation of spectral peaks

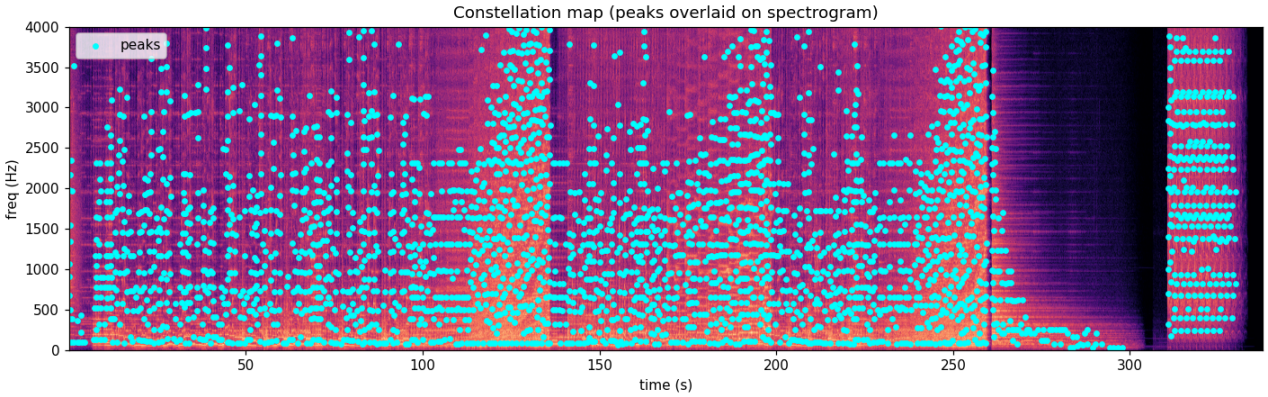

Fig 3. The constellation - the strongest local-maximum peaks - overlaid on the spectrogram.

From the spectrogram we keep only the strongest PEAKS: points that are a local maximum within a neighbourhood (a 20-bin maximum filter is compared against the spectrogram) and that exceed an amplitude floor. These sparse, high-energy points - the 'constellation' - are what survive distortion: even when noise or compression fills in the quiet regions, the loudest peaks remain in place. Discarding everything else also makes the representation compact.

4. Hashing: pairing peaks into compact, specific keys

A lone peak frequency is not distinctive - the same note recurs in countless songs. So each ANCHOR peak is paired with several nearby peaks lying ahead of it in a time-frequency 'target zone' (here, a forward time gap of 1-40 spectrogram frames, up to 15 partners per anchor). Each pair becomes a compact hash key (f1, f2, dt) - two frequencies and the time gap between them - with value (song, t_anchor). Building this for every song yields one inverted-index database mapping each hash to the list of (song, time) where it occurs. A joint triple (f1, f2, dt) is far rarer than a single frequency, so it is far more SPECIFIC.

5. Matching by time-offset alignment

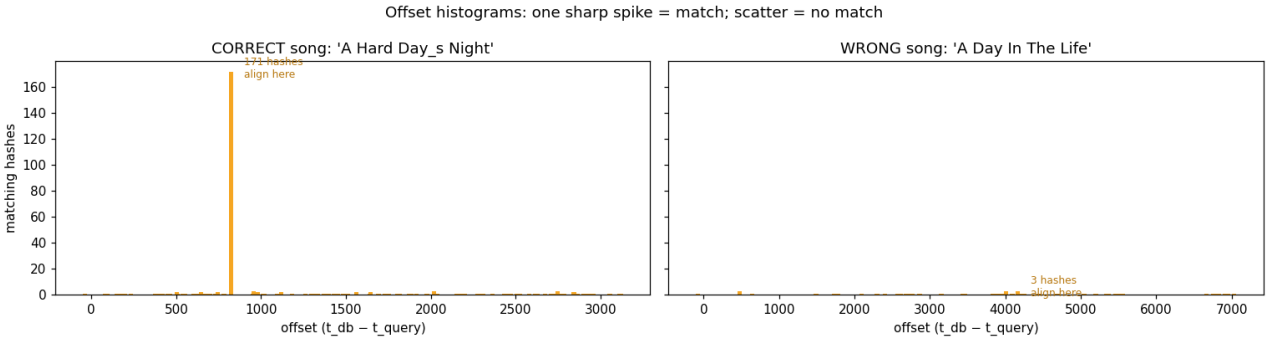

Fig 4. Offset histograms: the correct song's matched hashes pile up at ONE time offset (a sharp spike); a wrong song gives only scattered counts.

A query clip is fingerprinted the same way. For every query hash that hits the database, we record offset = t_database - t_query for the matching song and accumulate these offsets into a per-song histogram. The reasoning: if the query truly comes from a song, ALL of its matching hashes occur at the same fixed lag into that song, so their offsets coincide and the histogram shows one tall spike. For a wrong song, the few coincidental hash matches occur at random lags, so the histogram is a low, flat noise floor. The predicted song is the one with the tallest spike, and the spike height (divided by the runner-up's) gives a confidence. On the real library a 4-second query taken from 'A Hard Day_s Night' is identified correctly with a confidence of ~34x the runner-up; every built-in sample clip is identified correctly.

6. Single peaks vs. paired hashes - why pairing wins

Repeating the match using SINGLE peak frequencies as the key (no pairing) produces a much noisier, flatter offset histogram even for the correct song. The reason is specificity: an individual frequency bin

recurs constantly across unrelated songs, so single-peak matches collide by chance very often and the true alignment is buried in noise. Pairing constrains (f1, f2, dt) jointly, a coincidence that is far rarer, so the correct alignment dominates. Quantitatively, on the same clip the paired matcher is roughly an order of magnitude more decisive than the single-peak matcher (e.g. ~500x vs. ~30x winner-to-runner-up ratio in the app's live comparison) - which is exactly why joining two peaks into one fingerprint makes a correct match so much more confident.

7. Robustness experiments

Fig 5. Correct-song match score as additive white noise increases (SNR decreasing left to right).

ADDITIVE NOISE. Adding white noise at decreasing SNR and re-identifying, the match score degrades GRACEFULLY and recognition survives surprisingly heavy noise before collapsing past a breakdown SNR. This robustness is a direct consequence of the peak constellation: the loudest spectral peaks remain the loudest until the noise is strong enough to bury them, so the fingerprint is largely intact at moderate noise.

PITCH SHIFT / TIME STRETCH. Shifting the query up in pitch by even a small amount, by contrast, DEFEATS the identifier almost immediately (on the real library a +0.5-semitone shift already causes a wrong prediction). The reason is fundamental: the hash keys store ABSOLUTE frequency bins, and a pitch shift multiplies every frequency by a constant, moving every peak into a different bin - so essentially no hash collides with the database any more. A human, who perceives RELATIVE pitch relationships, hears the same song, but the absolute-frequency fingerprint no longer matches. (A pure time stretch similarly disturbs the dt component of the hashes.)

A FIX. Quantise peaks onto a LOG-frequency / constant-Q axis: there a pitch shift becomes a constant ADDITIVE offset that can be tolerated or searched over, instead of a multiplicative scaling that scrambles every bin. Alternatives are to allow a small +/- tolerance in the hash lookup, or to pre-index a few pitch-shifted copies of each song offline.

Q3B - 'Zapptain America': the identifier app

The identifier is wrapped in a deployed, interactive web application (custom dark-themed Flask backend + HTML/CSS/JS frontend) that reuses src/q3_fingerprint.py with no duplicated logic. It indexes the song library once into the shipped hash database so it works immediately on load.

Required modes

SINGLE-CLIP MODE (Identify tab): the user uploads a query clip (or records one live from the microphone, or picks a built-in sample) and the app displays, in order, the intermediate steps the brief asks for - the spectrogram, the constellation of peaks, and the offset histogram that decides the match - followed by the predicted song name and a confidence score, plus a per-stage timing breakdown and the ranked candidate songs.
BATCH MODE (Batch tab): the user uploads a set of query clips; each is identified against the indexed library and the results are written to a standardised results.csv with EXACTLY two columns, filename and prediction, where prediction is the matched track's filename without extension (or 'none' when no candidate clears the confidence threshold). This exact format was verified for the automatic grader.
LIBRARY MODE: shows the whole indexed library as cards (each with its constellation thumbnail and hash count).

Extra analysis exposed in the app

Beyond the brief, the app makes the Q3A analysis interactive: a Robustness Lab runs the noise-SNR and pitch-shift sweeps on the user's own clip and charts how the score holds up, and a single-peak-vs-paired-hash panel shows the two offset histograms side by side on the live query so the specificity argument of Section 6 can be seen directly.

Deployment

The app is deployed on Hugging Face Spaces (Docker), which provides ample memory for the librosa/numba stack and avoids cold-start; the indexed database ships with the app so it is usable immediately. Both modes were tested end-to-end on the live link.

LIVE APP: https://mayur2403-ee200-audio-fingerprinting.hf.space
SOURCE CODE: https://github.com/Mayurdp2403/ee200-course-project